DHAR

Bare Metal. Absolute Control.

VOLUME IV — PROGRAM MANAGEMENT

THE PACT OF DEVELOPING DHAR

"A pact with the project's own future work: what counts as done, and who owes what."

DOCUMENT	The Pact of Developing Dhar
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume IV (the whole of Volume IV)
DEPENDS ON	Every prior volume — this document synthesizes, it does not originate; Amendments A-1 and A-2
AUDIENCE	Leadership, contributors

1. Mandate

Collect the roadmap, the risks, and the success metrics that the rest of this Canon already produced piece by piece, into one place accountable for tracking them as a whole. Every item below is a cross-reference to a claim ratified elsewhere, not a new claim invented here — The Pact's job is synthesis and follow-through, not authorship.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). For the roadmap that gives two lines of work: the five stages of Volume II, which build the compiler and its library on every committed target, and the gates of the three tracks, which say when each track's claims may be made.

2. Roadmap, in Dependency Order

Stated as a dependency sequence, not a calendar — this Canon does not fabricate target dates it has no basis to commit to; each stage's own document (Volume II) states exactly what must be true before the next one may begin.

ORDER	MILESTONE	GATE TO THE NEXT MILESTONE
1	Stage-alpha compiles the bootstrap subset for every committed target	Every committed target has passed gate G1 and Stage-alpha compiles Stage-beta's complete source (Stage-alpha §4)
2	Stage-beta is self-hosting	Byte-for-byte identical generation-2 rebuild (Stage-beta §4) — the one criterion that matters — together with the two-compiler check of gate S4 and a passing back end for every committed target (Stage-beta §4.1, §4.2)
3	Stage-gamma reaches a minimum viable optimizer	SSA + register allocation + dead-code elimination landed (Stage-gamma §5), each pass with its constant-time test (Stage-gamma §3.1)
4	Stage-delta: Kun and the standard library work	Every acceptance criterion in Stage-delta §7, including a demonstrated interop example per language in §4
5	Stage-epsilon: production-ready	Reproducible, signed, cross-compiled, SBOM'd releases that pass the release gates (Stage-epsilon §3.2, §4)

Volume III's six documents do not gate Volume II's engineering sequence and were written in parallel deliberately — trust, adoption, and sustainability work starting late is a slower failure than starting before Stage-alpha exists, not a safer one.

The work of the three tracks begins when milestone 2 is reached and runs beside milestones 3 to 5, in the order cybersecurity, artificial intelligence, quantum computing (Amendment A-1 §7). Nothing in the tracks begins earlier. The gates of the tracks are the milestones of §2.2.

2.1 The order of the targets

The order in which the committed targets (Amendment A-2 §3.1) are built is a roadmap matter, and it lives here (Amendment A-2 §7). Two dependencies are fixed by that amendment: the neutral operation set and the first prelude come before any other target, and the WebAssembly target comes before the in-browser playground (Tooling Dhar §5). The rest of the order is chosen so that what is easiest to test on real systems comes first and what is hardest comes last:

ORDER	TARGET	REASON FOR ITS PLACE	WHAT GATE G1 NEEDS
0	Linux x86-64, the host	Stage-alpha is a program for this target and runs on it	A Linux x86-64 machine
1	The neutral operation set, then Linux ARM64	The first prelude is the test of the neutral operation set; Linux ARM64 uses a raw system-call interface like the host, so little else changes	An ARM64 machine running Linux
2	Linux RISC-V 64	The same style of interface as the target before it; its short branch range tests the prelude rule of Stage-alpha §3.1	A RISC-V 64 machine running Linux
3	WebAssembly with WASI (preview 1)	Structured control flow needs the dispatcher form of Stage-alpha §3.1; a WASI host is a real runtime for gate G1; the playground needs it	A WASI runtime, and a browser for the playground
4	Windows x64	A different program format (PE32+) and the operating system's own library as the interface	A Windows x64 machine
5	macOS x86-64 and macOS ARM64	Last, because gate G1 needs Apple hardware, and because native programs on Apple silicon must carry a code signature that Dhar has to produce without an outside tool (Amendment A-1, P1)	An Apple computer of each kind

The order changes by a revision of this section, not by an amendment to Amendment A-2. A target that cannot reach gate G1 is neither described as supported nor counted as complete; if it cannot reach it, Amendment A-2 §3 is revised to move it between the committed and the candidate list, and Stage-beta's start is decided by the list as revised (§3, risk on access to hardware).

2.2 The gates as milestones

A milestone of the three tracks, or of a target, is reached when its gate passes, never when the work is judged finished (§4). The gates are defined in Amendment A-1 §9 and Amendment A-2 §6; the order in which work on them proceeds, and what each needs first, is Amendment A-1 §7. This table lists them in the order of the tracks, and nothing more.

GATE	MILESTONE REACHED WHEN	TRACK
G0, G1	A committed target is enabled in the compiler (G0); a committed target is complete on a real system (G1)	Targets, before Stage-beta
S1	The memory-safety floor holds on the suite of rejected and trapping programs	Cybersecurity

GATE	MILESTONE REACHED WHEN	TRACK
S2	Programs that branch on, index with or divide by a secret are rejected, and optimized output has no secret-dependent branch or access	Cybersecurity
S3	The first `std.crypto` passes its published vectors and the timing test	Cybersecurity
S4	The compiler builds the same bytes twice and on a second machine, by two independent compilers, with a complete bill of materials	Cybersecurity
A1	Numeric edge cases are bit-identical to the reference results	Artificial intelligence
A2	Tensor and view misuse is rejected or traps, and an arena's size is known before the run	Artificial intelligence
A3	The reference network agrees with a reference framework to a stated tolerance	Artificial intelligence
A4	The ratio to optimised C on matrix multiplication and the reference network is measured and published	Artificial intelligence
Q1	The post-quantum algorithms pass the NIST known-answer tests	Quantum computing
Q2	Circuits written out as OpenQASM 3 are accepted by a reference tool and simulate to the expected result	Quantum computing
Q3	The state-vector simulator agrees with a reference simulator up to its stated limit	Quantum computing
Q4	Not defined until Q2 and Q3 have passed	Quantum computing

A module outside the three tracks has no gate in this table. It is not held back for that reason, and it is not placed ahead of a track's gate (Amendment A-1, P6). A gate is added to this table only by an amendment that adds it to Amendment A-1 or A-2.

3. Risk Register

RISK	WHERE IT'S NAMED	MITIGATION OF RECORD
A Thompson-style compiler backdoor surviving self-hosting undetected	Tempering Dhar §5.3	Diverse Double-Compiling, specified as gate S4 and as a release gate (Tempering Dhar §5.3; Stage-epsilon §3.2); what remains is the frozen Stage-alpha, the toolchain that builds it and the machine it runs on, which the check does not cover
Single-maintainer bus factor	The Guild of Dhar §6, Fueling Dhar §4	Foundation-transition trigger tied to funding or a multi-employer contributor base, not a calendar date
Funding runs out before Stage-epsilon	Fueling Dhar §3	Sovereign Tech Fund and comparable infrastructure-focused funders, applied for against scoped deliverables, not general support

RISK	WHERE IT'S NAMED	MITIGATION OF RECORD
Spec1 scope creep undermining Spec0's simplicity commitment	Observed directly in this Canon's own history: 37 keywords (Spec1 v1.0) grew to 46 (v3.0) inside one revision cycle, and four provisional words followed in v3.2	Every future Spec1 addition must go through The Guild of Dhar §3's RFC process, including the explicit alternative-considered-and-rejected discipline, rather than being added ad hoc
Adoption strategy depends on institutions Dhar does not control	Casting Dhar §3-4 (3MTT, Smart Africa)	Curriculum content designed as a portable module (Casting Dhar §2.1), not wired irreversibly into one program's specifics
Hardware/connectivity constraints silently excluding the primary target users	Casting Dhar §3.2	Kun's offline-cache requirement (Stage-delta §3.3, added specifically for this) tracked as a release-blocking property, not an optimization
Premature or overstated certification claims damaging credibility	Tempering Dhar §7, §11	The rule that a claim names its gate or is worded as an aim (Tempering Dhar §11), audited by whether documents, releases and the website keep to it
Export-control exposure from std.crypto, the post-quantum algorithms or a defence-specific build	The Hallmark of Dhar §5	Mandatory legal review triggered at std.crypto's first releasable state, not deferred indefinitely
The cost of a generator for every target in the first compiler	Amendment A-2 §1.2	The neutral operation set and one prelude per target (Stage-alpha §3.1); Stage-alpha's output becomes the oracle for Stage-beta's back ends (Amendment A-2 §7)
A target that has only run in an emulator mistaken for a finished one	Amendment A-2 §6	Gate G1 requires a real system; a statement of support names the gate and the run (Tempering Dhar §11)
Access to real hardware for gate G1 (RISC-V, Apple computers, Windows) delaying Stage-beta, which waits for every committed target	Amendment A-2 §2, §6; §2.1 above	The order of §2.1 puts the targets that are hardest to reach last; a target that cannot reach gate G1 is moved between the committed and candidate lists by revising Amendment A-2 §3, never counted as complete
The WASI interface changing under the WebAssembly target	Amendment A-2 §3.1	The target is WASI preview 1; a later interface is a new target, added by revising Amendment A-2 §3
Principle P1 read as forbidding the operating system's own library on Windows and macOS	Amendment A-2 §3.1	P1 is about the language, the compiler and the standard library; the library is named as that target's interface (Tempering Dhar §5.4)
Breadth (P6) diluting the three tracks, or the tracks narrowing breadth	Amendment A-1, P6	The tracks decide the order of work; a module is never held back for lack of a gate and never placed ahead of a track's gate (§2.2)

RISK	WHERE IT'S NAMED	MITIGATION OF RECORD
A constant-time property lost by a later change	Tempering Dhar §6.1	The timing test in continuous integration (Stage-epsilon §3.2) and a test per optimization pass (Stage-gamma §3.1)
Stage-alpha and Stage-beta back ends diverging	Amendment A-2 §7	Both pass one conformance battery; Stage-alpha's output is the reference (Stage-beta §4.2)
A defect found in Stage-alpha after it is frozen	Tempering Dhar §8	A repair is a new archived version with its own recorded hash, named in the advisory; the archive is never edited silently
Collecting personal data (mailing list, survey, playground) without a sound legal basis	The Hallmark of Dhar §7	Counsel qualified in Nigerian law reviews the notice and the process before any collection begins

4. Success Metrics

Engineering metrics (this Canon's own Volume II acceptance criteria are the primary record) plus the adoption metrics Casting Dhar §6 proposed, formalized here as the set this Canon actually tracks going forward:

- Stage completion against Volume II's acceptance criteria — binary, per stage, not a percentage estimate.
- Gates passed against Amendment A-1 §9 (S1 to S4, A1 to A4, Q1 to Q4) — binary per gate, each with the run and the commit that passed it.
- Committed targets at gate G1 (Amendment A-2 §6), out of the list in Amendment A-2 §3.1 — counted by gate, never by a program having run once.
- 3MTT fellows completing the Dhar module, and how many publish a Kun package afterward (Casting Dhar §6).
- Non-trivial published Kun packages and their sustained (not launch-week) download counts.
- Countries with an active local user group, against Smart Africa's member-state list as the honest ceiling.
- Government or institutional pilot deployments — the metric this Canon weights most heavily, since it is the one that validates Spec0 §5 rather than merely gesturing at it.
- Funding secured against scoped deliverables (Fueling Dhar §3), tracked separately from general goodwill or interest.

5. How This Document Stays Honest

The Pact is reviewed whenever a Volume II stage completes, a Volume III document is amended, or an amendment is ratified (Master Index §10), specifically to catch exactly the failure mode §3's fourth risk names: a Canon that keeps growing in one place without anyone checking whether an earlier promise quietly stopped being true. A risk in §3 that has been mitigated is marked closed with the evidence, not deleted; a new risk discovered during any stage's work is added here rather than left implicit in that stage's own document.

The Pact contains no status. Which milestones of §2 and §2.2 have been reached is reported in the Progress Reports and on the website's status board, which are generated from one record and are outside the Canon (Master Index §2). A milestone is reached when its gate passes, and a statement never reaches it (Tempering Dhar §11).

Revision log

The Pact of Developing Dhar · v1.1 · October 26, 2026 · Ratified. This document completes the initial ratification of all four Volumes of the Canon of Dhar.

VERSION	DATE	CHANGE
v1.0	September 9, 2026	first ratified edition, synthesizing roadmap, risk, and metrics content from Spec0, Spec1, all five Volume II stage documents, and all six Volume III documents.
v1.1	October 26, 2026	Amendments A-1 and A-2: the roadmap restated with every committed target at gate G1 (§2), the order of the targets (§2.1), the gates as milestones in the order of the tracks (§2.2), risks added or restated (§3), and metrics counted by gate (§4).